WEEK 02 · BLOCK 2B

Block 2b — Build a registration form

Goal: build a real registration `<form>` with varied inputs, a `<label>` on every field, and native validation — then watch exactly what the browser **sends** on submit.

Time: ~45 minutes · **You need:** a browser and a terminal (Node from Block 0)

Setup

```
cd exercises/week02/2b/starter
npm install     # once
npm start       # serves at http://localhost:3000
```

Open http://localhost:3000, edit `index.html`, **save**, **reload**. The form already has one example field (email) — copy its shape for the rest.

Task 1 — Read the example field

Find the email field in `index.html` and make sure you can name every part:

- the `<label>` and its `for` — how does it link to the input?
- the input's `id`, `name`, `type`, and `required`
- Which attribute links label ↔ input? Which one becomes the **key** when you submit?

Click the *word* "Email" (the label) — the input should focus. That's the label doing its job.

Task 2 — Add the fields

Replace each `TODO` comment with a real field. **Every input gets its own `<label>`.**

Field	Type	Notes
`username`	text	`required`, `minlength="3"`, `maxlength="20"`
`password`	password	`required`, `minlength="8"`
`age`	number	`min="16"`, `max="120"`
`birthday`	date	—
`country`	`<select>`	a few `<option>`s (give each a `value`)
`plan`	radio ×2	two options *free* / *pro* that **share** `name="plan"`
`terms`	checkbox	`required` — "I accept the terms"

Task 3 — See what the form sends

1. Fill the form in and click **Sign up**.
2. Look at the **URL**: it now ends with a **query string** — `?email=…&username=…&age=…`
   - Find each field's `name=value`. Do the values match what you typed?
   - The radio sends **one** `plan=…`. The checkbox only appears **if ticked**.
3. Open the **Network tab**, submit again, click the request, and find the **query string parameters** listed there too.

> This is the request from Block 1 — and *you* built the thing that generates it.

Task 4 — Watch validation work

- Submit the form **empty**. What happens? Which field does the browser complain about first?
- Type `hello` (no `@`) into email and submit. What message appears?
- Enter `10` into age. Does it let you submit?

All of that is **native validation** — no JavaScript. It blocks the submit and guides the user.

Task 5 — The catch (think about it)

You just saw the browser enforce `required`, `type="email"`, `min` … Now:

- Open dev tools → **Elements**, find the email input, and **delete** its `required` attribute live. Submit empty. It goes through.
- **Question:** if a user can do that in two clicks, can the server trust that the data it receives obeyed *any* of your rules? Write one sentence: what does this mean for where **real** validation has to happen? `______`

(We answer this properly in Block 7a. For now: native validation = UX, never security.)

Requirements checklist

- [ ] A `<form method="get">` containing a `<fieldset>` + `<legend>`
- [ ] All fields from the Task 2 table, each with a **paired** `<label>`
- [ ] Correct `type` on each input; a `name` on every field
- [ ] Validation: `required` where listed, plus `minlength` / `maxlength` and `min` / `max`
- [ ] Submitting shows every value in the URL's query string
- [ ] Empty / invalid submits are blocked by the browser

At home

Do this task after class. It continues the numbering, so you can ask about it by number.

Task 6 — Send it with POST

So far your form sends its data in the URL. Now send it in the **request body** instead.

1. In `index.html`, change the form to `method="post"`.
2. Fill the form in and click **Sign up**. Look at the **URL**: is there still a query string? `______`
3. Open the **Network tab**, submit again, and click the request.
   - Which **method** does it use? `______`
   - Open the **Payload** panel. Where is your data now, and what is it called there? `______`
4. Which method would you use for a **login** form, and why? Is the password safe with `POST` alone? `______`

> After the submit, the form is empty again. That's expected: the small server from `npm start` only sends files and ignores the data. In Block 7 you write a server that reads it.

Stretch goals (optional)

- Add a `placeholder` to one field — then confirm it is **not** a replacement for the label (it disappears as you type).
- Add a second `<fieldset>` (e.g. "Profile") to group optional fields.